

Swarm Intelligence

Report on Assignment 3

Aaron Bussche

David Magaram

Jason Kasdiran

Group 28

March 9, 2026

1 Part A

1.1 Introduction

An image can be compressed by reducing the number of distinct colors in it. This process is called color quantization. The objective of finding the optimal color palette for a given image can be seen as a clustering problem. In this assignment, we tackle the color quantization problem using Particle Swarm Optimization (PSO). We implement standard PSO to iteratively optimize image color palettes and then evaluate its performance by comparing the resulting quantization against a popular clustering method, K-Means.

1.2 Preliminary Concepts

To process the image digitally, a color is represented numerically as a 3-dimensional vector corresponding to its Red, Green, and Blue (RGB) channel intensities, and a target color palette is represented as a 2D matrix of shape $(N, 3)$, where N is the number of available colors in the palette.

To quantize an image using a given palette, we implemented a custom function to evaluate every pixel in the input image array. For each pixel, it computes the squared Euclidean distance between the pixel's RGB vector and each of the N candidate color vectors in the palette. The original pixel is then replaced by the closest color in the given palette. Testing this function on the provided image and our own image using the pre-defined 4-color palette successfully reproduced the correct target quantization (Appendix 1).

To find an optimal color palette for an image we approached it as a clustering problem and used the popular K-Means clustering algorithm. In this clustering context:

- The **data** points represent the individual pixels of the original image, defined by its (R, G, B) values.
- The clusters represent groupings of pixels with similar RGB values. The **centroid** of each cluster acts as the representative color for all pixels assigned to that cluster. Therefore, the list of the N optimal cluster centroids directly forms the optimal N -color palette.

By clustering the pixels, the goal is to minimize the intra-cluster variance, which in this case equates to minimizing the Mean Squared Error (MSE) between the original image pixels and the restricted colors of the new palette.

1.3 Methods

To map the color quantization problem to PSO, each particle in the swarm represents a candidate solution, which in this case is a list of N cluster centroids. Mathematically, a particle's position is defined as $x_i = (m_{i1}, \dots, m_{iN})$, where each centroid m corresponds to a color in the palette.

We measure the quality or "fitness" of a particle by computing an evaluation function $f(x_i, Z)$ over the image dataset Z containing M pixel points. During the evaluation, each pixel z_j is assigned to the nearest centroid m . The implementation maps the input image to the color palette by finding the minimum Euclidean distance between each pixel and the candidate colors, outputting the reconstructed

image to calculate the Mean Squared Error (MSE). The PSO algorithm stores the state of the particles and the global best particle at each iteration, allowing the swarm to minimize the MSE over time.

1.4 Experimental Setup

The PSO clustering algorithm was implemented and evaluated on four test images: "peppers", "colorful", "sunrise pixel art", and "very small". Because the PSO algorithm can take very long to evaluate fitness on large datasets, the "colorful" image was resized from 1328×2000 to a smaller resolution of 85×128 prior to running the optimizer.

For the optimum target color palette we tested 2 different sizes. We tested $N = \{4, 8\}$ colors color palette. The PSO swarm consisted of 30 particles, and the algorithm was executed for a maximum of 100 iterations. To benchmark our implementation, the standard K-Means algorithm (via `sklearn.cluster.KMeans`) was applied to the same images with $K = \{4, 8\}$. Both algorithms were quantitatively compared based on the final MSE of the quantized image.

1.5 Results

The PSO implementation successfully quantized all four images to a N -color palette. In the appendix we make side by side comparisons of the original, the PSO quantized and K-Means quantized images. Table 1 presents the final MSE values for PSO and K-Means across all images and both palette sizes.

Image	PSO $N = 4$	K-Means $K = 4$	PSO $N = 8$	K-Means $K = 8$
Peppers	524.25	524.26	251.34	222.87
Colorful	1373.19	1358.31	728.12	693.26
Pixel Art	757.08	742.91	169.50	160.44
Very Small	4324.48	4565.67	365.11	244.93

Table 1: Final MSE values comparing PSO and K-Means with N colors color palette

For $N = 4$ colors, PSO matched or outperformed K-Means slightly on the Peppers and Very Small images, while K-Means achieved a lower MSE on the Colorful and Pixel Art images. With $N = 8$ colors, K-Means consistently achieved a lower MSE on all images except Peppers where PSO was marginally better. Table 2 shows how the PSO global best MSE evolved across 100 iterations on the Peppers image.

Iteration	PSO $N = 4$	PSO $N = 8$
10	603.20	327.29
20	571.08	298.02
30	543.23	283.06
40	537.82	264.56
50	530.71	257.80
60	526.01	255.29
70	524.75	252.35
80	524.36	252.16
90	524.26	251.73
100	524.25	251.34

Table 2: MSE values at every 10 iterations of PSO on the Peppers image.

The convergence behaviour shown in Table 2 indicates that PSO reduces MSE rapidly in the early iterations (iterations 10–50) and then slows considerably as the swarm approaches the global best. By iteration 90, the improvement became insignificant, suggesting the algorithm had effectively converged before the 100-iteration limit was reached.

1.6 Discussion

The results confirm that Particle Swarm Optimization can successfully be applied to color quantization and is capable of finding optimal, or near-optimal, color palettes. Overall, K-Means achieves a lower or equal MSE compared to PSO in the majority of cases. This is expected, as K-Means is guaranteed to converge to a local optimum through exact centroid reassignment, while PSO is a stochastic method with no such guarantee. The exception is the Very Small image at $N = 4$, where PSO outperformed K-Means by a significant margin, likely due to K-Means being sensitive to poor initialization on small datasets, where PSO’s broader exploration is beneficial. In contrast, for the Pixel Art image, which contains smooth gradients and many varying hues, K-Means produces a noticeably better quantization. This suggests that K-Means benefits from images with complex, continuously distributed color spaces, where its exact centroid updates efficiently capture fine color variation, while PSO’s exploratory search offers an advantage when the color space is sparse and initialization-sensitive.

A key practical limitation of PSO is its computational cost. The Colorful image had to be downsampled to 85×128 pixels before running PSO, while K-Means could handle the full resolution efficiently. Additionally, PSO’s stochastic nature introduces run-to-run variability, and the hyperparameters were not systematically tuned, meaning results could vary with different configurations.

1.7 Conclusion

PSO proved to be a viable approach to color quantization, converging consistently across all tested images. However, K-Means generally produced lower MSE values, particularly for larger palettes, due to its exact and efficient centroid optimization. PSO showed competitive performance mainly in cases where K-Means is prone to poor initialization.

2 Part B

2.1 Introduction

The Boids model from Craig Reynolds is one of the most popular models of swarm intelligence and group behavior. It uses boids, which are bird like objects, that follow 3 simple rules: alignment, cohesion, and separation. These simple rules govern the interactions between individual boids, which gives rise to a more complex emergent behaviors such as flocking. This is also known as swarm intelligence. Swarm intelligence is some collective behavior that emerges when individual agents interact with each other. This can lead to flocking, fish schools, and insect swarms. The goal of this project is to implement the Boids model and explore how the swarm behavior emerges from simple rules and how modifying the parameters affect it.

2.2 Methods

The JavaScript framework was used to implement the two-dimensional Boids model. The model was defined on a square grid of 400×400 pixels with wrapped dimensions. 150 boids were randomly initialized, each moving at a constant speed. The position and direction of each boid were updated at each time step based on the three rules, and the current boid was not included in the calculations. The simulation was ran for 300 time steps and the used parameters in this project are displayed in Table 3 below.

Boids follow three simple interaction rules: alignment, cohesion, and separation. Alignment controls how each boid moves towards the average direction of the other boids nearby. Similarly, cohesion controls how quickly each boid tries to move towards the average position of the other boids around. Separation dictates how much boids try to keep a certain distance from each other to avoid collision. Together, these rules dictate how boids interact with each other. However, other than these 3 basic behavior rules, each boid has an inner and outer radius. The outer radius is associated with cohesion and alignment, while the inner radius is linked to separation. The outer radius is the boids’ visual and interaction range. It is the distance within which the boid can see other boids and interact with them, and is visualized as a gray circle surrounding the boid. The inner radius, depicted by a red circle around the boid, prevents collision and is the opposite of the interaction range. If a boid enters

another boids inner radius, they are repelled away, controlled by the separation weight. For obvious reasons, the outer radius should be larger than the inner radius so cohesion and alignment can occur.

Parameter	Value
$Radius_{\text{inner}}$	10
$Radius_{\text{outer}}$	25
Alignment weight	1
Coherence weight	1
Separation weight	1
Number of boids	150

Table 3: Parameters.

To analyze the swarm, the order and nearest-neighbor quantitative metrics were used. Order parameter is the average normalized velocity of the Boids. Order parameter tells us to what degree the swarms align in their directions, and the output is squashed between 0 and 1, 0 meaning completely random directions and 1 meaning perfect alignment.

$$O = \frac{1}{N_B} \left\| \sum_{i=1}^{N_B} \frac{\vec{v}_i}{\|\vec{v}_i\|} \right\|$$

The nearest-neighbor distance is the average nearest-neighbor distance across all boids. It measures the total average distance across all boids and can tell us when individual boids get separated from the swarm or when individuals get too close to each other.

$$d_i = \min_{\substack{j=1 \\ j \neq i}}^{N_B} \|\mathbf{x}_i - \mathbf{x}_j\|$$

$$\bar{d} = \frac{1}{N_B} \sum_{i=1}^{N_B} d_i = \frac{1}{N_B} \sum_{i=1}^{N_B} \min_{\substack{j=1 \\ j \neq i}}^{N_B} \|\mathbf{x}_i - \mathbf{x}_j\|$$

2.3 Results

Visuals. Quantitative results. Parameter experiments.

Is there swarm alignment? How does alignment change over time? How do boids maintain spacing? How do parameters influence behavior?

It is important to note that due to the random initializations the results can greatly vary between runs.

It is possible to reach high alignment in 300 steps with these parameters.

The visualizations in Figure 1 show the positions of the boids at different time steps. Four snapshots were taken at time steps 0, 100, 200, and 300, and are separated into an initial state, final state, and two intermediate states.

Red color represents the inner radius, while the black bar coming out of it indicates in which direction each boid is facing. The outer radius is shown as a gray circle and is each boids' interaction range.

150 boids were initialized randomly on a 400x400 field with wrapped boundaries. Boids then move around, updating their positions and orientations based on the alignment, coherence, and separation rules. Boids interact with each other by coming in each other interaction range, forming flocks as they align and cohere with each other, while also maintaining a certain distance to avoid collisions. Over time, larger flocks form as boids and smaller flocks join together. However, as shown in Figure 1d, not all boids are able to find a flock by time step 300 and remain isolated until they bump into another boid or flock.

The boids are distributed across the simulation field, and their positions and orientations change over time as they interact with each other based on the alignment, coherence, and separation rules.

Over time, we can observe the formation of flocks as the boids align and cohere with each other. The separation behavior can also be seen as boids maintain a certain distance from each other to avoid collisions.

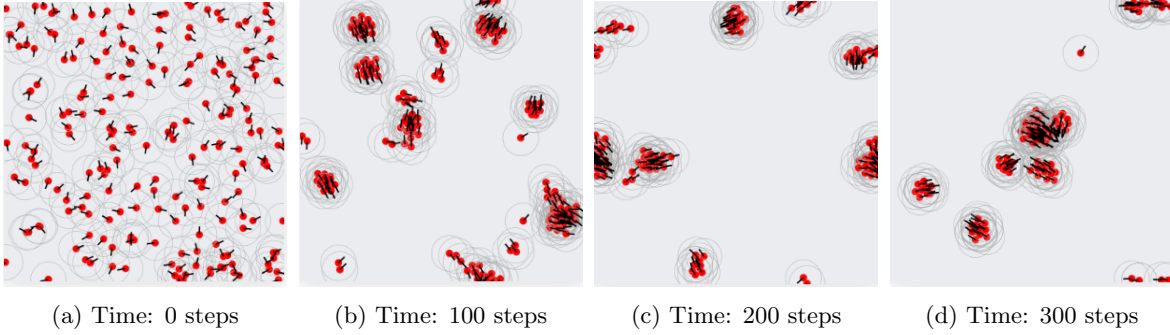


Figure 1: Visualizations of boids at four different time steps using default parameters.

Table 2 accompanies the visualizations from Figure 1 and displays the order and nearest-neighbor distance values obtained at the initial, intermediate, and final states. For further analysis, Figure 2 plots display the order parameter and nearest-neighbor distance values against time. A pattern can be observed where order grows and nearest-neighbor shrinks as time steps increase. This is expected as smaller flocks that formed during the initial phase wander around and ultimately bump into each other with others, merging and creating a large unified flock.

Time	0	100	200	300
Order parameter	0.0208	0.0753	0.105	0.616
Nearest-neighbor distance	13.956	5.925	4.278	3.405

Table 4: Quantitative results for default parameters.

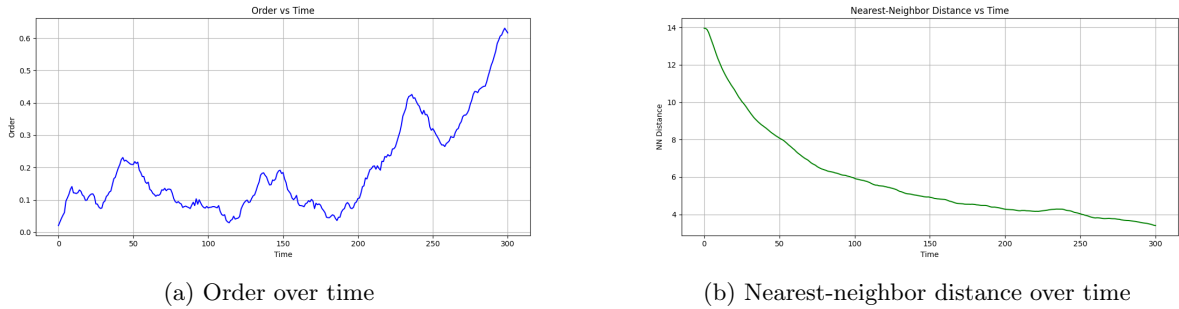


Figure 2: Plots of quantitative results over time.

Parameter	Value
$Radius_{inner}$	5, 10, 20
$Radius_{outer}$	25, 40
Alignment weight	0, 0.33, 0.66, 1
Coherence weight	0, 0.33, 0.66, 1
Separation weight	0, 0.33, 0.66, 1
Number of boids	150

Table 5: Explored parameter configurations.

Criterion	R_{in}	R_{out}	Align	Cohes.	Order	NN-dist
Best Order	20	40	1	0	0.999	14.36
Best NN-dist	20	40	0.66	1	0.305	0.03
Best Ratio	5	40	0.33	1	0.625	0.05

Table 6: Parameter configurations that maximize each criterion at $t = 300$, selected from 384 configurations ($N=150$, separation=0, averaged from 3 repeats each). Order and NN-dist columns show the resulting metrics.

From the results of Table 6 it appears that to maximise order, it is important that the boids have a big outer sight radius and a high alignment parameter value. To minimise Nearest-Neighbour-distance, they need a high cohesion value, though unexpectedly the inner radius was still quite large at 20. Lastly, the best ratio of order/NN-distance was achieved with a low inner radius and a smaller alignment value of 0.33, and largely by lowering the NN-distance, not by maximising the Order.

2.4 Discussion

Why behaviors emerge. How the parameters affect the behaviors. What are the limitations of the model? How to improve it?

Be sure to come to a final conclusion and discuss some potential limitations as well.

Assuming this project’s code implementation is correct, adjusting the parameters gives expected results. For example, increasing the outer radius gives high cohesion and very short time because boids have a much larger interaction rate. On the other hand, the inner radius should generally be smaller than the outer radius for interactions to happen. A rule of thumb is to have the outer radius be at least twice the size of the inner radius. Low alignment weight causes the boids to spin around. Low separation weight results in low order parameter values as boids do not create flocks but enter each other, making their flock’s total interaction radius much smaller.

2.5 Conclusion

3 Appendix Part A

3.1 Preliminary Concepts

Color quantization on the picture of peppers given a color palette.

Color quantization on a colorful picture using the same color palette from figure 1.

Color quantization using K-Means and finding the optimal color palette (centroids).

Color palette from figure 3.

3.2 Results